

Patient Chat and Voice Scheduling Web App for a Physician Group

What to build first and what to defer

The fastest path to a “complete-feeling” product is to constrain the AI to a small set of workflows, make scheduling feel flawless, then add only the lightest supporting workflows that round out the experience. The app should behave like a modern concierge, it can speak naturally, but it is always steering toward structured outcomes.

For an MVP that hits the evaluation criteria you described, the required end user flows are:

1) **Appointment scheduling (primary)** - Patient starts in a web chat UI. - AI offers clear choices early, for example “Book an appointment”, “Office hours and address”, “Prescription refill status”. - When booking, AI captures: first name, last name, date of birth, phone, email, reason for appointment. - AI asks for the **body part** and uses semantic matching to pick a provider, then proposes available times in the next 30 to 60 days. - Patient can ask constraints in natural language, for example “Do you have something on a Tuesday?”, and the AI filters accordingly. - Booking confirmation appears in chat, an email is sent automatically, SMS is sent only after opt-in.

2) **Voice handoff with full context (primary)** - A visible “Continue by phone” button initiates an outbound call to the patient’s number. - The voice AI continues the same conversation, with the same context and current booking state. - The app should also support an inbound call to the same number as a single extra feature, so if the call drops, the patient calls back and the AI resumes smoothly.

3) **Practice information workflow (secondary, fast to ship)** - Address, hours, parking details, and “what to bring” can come from hard-coded practice configuration and should be extremely reliable.

4) **Prescription refill workflow (secondary, minimal)** - Unless you integrate an EHR, treat this as “intake and status request”: collect medication name, pharmacy, best callback number, and send an internal “refill request” ticket. It must not attempt medical guidance.

The MVP should explicitly avoid medical advice, and should elegantly redirect. This is important for the voice safety testing and for healthcare risk controls in general. A strong pattern is: confirm you can help with logistics, state you cannot provide clinical guidance, and offer to connect them with the office or advise urgent care pathways if they indicate emergency symptoms. The US government’s HIPAA guidance highlights that identifiers like name and date of birth contribute to individually identifiable health information, which is why you should minimise what is stored and logged. ¹

UI expectations for “liquid glass”

“Liquid glass” in practice usually means glassmorphism plus modern motion. Nielsen Norman Group describes glassmorphism as layered translucency that mimics frosted glass, and it works best over gradients or complex backgrounds. ²

Use motion to make the system feel alive: subtle chat bubble entrance, typing indicator shimmer, button hover parallax, and a smooth transition when switching from chat to “Calling you now...”. Motion (formerly Framer Motion) is a common production library for this kind of effect in React. ³

Reference architecture that ships quickly and stays extensible

A practical structure is a three-layer system: UI, an application backend, then an “AI orchestration layer” that is mostly deterministic code with an LLM inside very explicit guardrails.

Suggested stack for speed and polish

- **Frontend:** Next.js App Router for a modern streaming UX and simple deployment packaging. Next.js supports streaming and progressive rendering concepts that map well to chat UIs. ⁴
- **Backend:** Node.js + TypeScript, either a dedicated API service (Fastify/Express) or Next.js route handlers, depending on team preference.
- **Data:** PostgreSQL for persistence (patients, conversations, appointments, SMS consent), plus Redis optionally for short-lived session state and rate limiting.
- **Integrations:** Email (AWS SES or SendGrid), SMS (Twilio), Voice (Vapi or Retell for fastest end-to-end, or Twilio Media Streams + OpenAI Realtime if you want more control).

The key product requirement is continuity between chat and voice, so the architecture should treat both channels as surfaces on top of a shared “Conversation Session”.

The core concept: a shared Conversation Session

Maintain a single server-side record per session: - `conversation_id` - `patient_profile` (structured intake fields, completion status) - `current_workflow` (scheduling, office info, refill) - `workflow_state` (step, selected provider, candidate slots, selected slot) - `conversation_summary` (short model-generated summary, updated each turn) - a small rolling window of recent messages (for immediate coherence)

When the user clicks “Continue by phone”, you start a voice session and pass in: - `conversation_id` - `conversation_summary` - `workflow_state` snapshot

This avoids passing full transcripts everywhere and makes it easier to keep voice and chat consistent.

Model and agent design that is state of the art and production friendly

The strongest pattern for your use case is “LLM plus tools plus strict structure”:

- The LLM’s job is to converse naturally, choose the right workflow, and call tools.
- Tools do all critical logic: slot searching, filtering Tuesdays, booking, sending email and SMS, starting a phone call, fetching office hours.
- Every external side effect is done by deterministic code, not by freeform model output.

OpenAI’s function calling guide describes the standard loop: provide tools, receive tool call, execute it in your application, then send tool output back for a user-facing response. ⁵

Structured outputs are useful when you want the model’s final response to follow a schema for UI rendering, while tools handle actions. ⁶

The OpenAI Agents SDK is explicitly designed to support agentic apps with tools, handoffs, and tracing, and the JavaScript/TypeScript package is positioned as provider-agnostic. ⁷

Recommended “agent composition”

Instead of one giant prompt, use small role agents behind a single UX:

- **Router Agent:** Determines which workflow to run (scheduling vs office info vs refill), based on the user message.
- **Scheduling Agent:** Drives the intake form completion and booking, it never gives clinical advice.
- **Voice Agent:** Same as Scheduling Agent but tuned for spoken turn taking, confirmations, and disfluencies.
- **Safety Policy Layer:** A classifier or rule-based gate that detects medical advice requests, urgent symptoms, or abusive content, and forces safe responses.

If you want a SOTA pattern without overbuilding, implement “policy then agent”: 1) Run a lightweight policy check on each user message, return a flag like `MEDICAL_ADVICE_REQUEST`, `EMERGENCY_SYMPTOMS`, `OK`. 2) If flagged, respond with a safe template and optionally offer a call transfer or “office will follow up”. 3) Only if OK, allow the workflow agent to proceed.

For AI risk framing, the National Institute of Standards and Technology provides an AI Risk Management Framework intended to help incorporate trustworthiness considerations into AI product design and operation, which is relevant for healthcare interactions. ⁸

Selecting models: GPT vs Claude vs Gemini

You can absolutely build this with any of the three, but the best practical plan is to choose one primary provider for the MVP and keep a provider abstraction.

OpenAI - Good fit if you want one vendor for chat, tool calling, and voice. - OpenAI’s model pages describe current flagship and tool support, for example GPT-5.4 is positioned for complex agentic workflows and supports function calling and structured outputs. ⁹
- For voice, OpenAI provides Realtime models for low-latency audio in and out, and separate guidance on voice agent architectures, including speech-to-speech and chained approaches. ¹⁰

Anthropic - Strong quality for dialogue and reasoning, tool use is first class. - Anthropic documents tool use as a primitive where the model decides when to call tools you provide. ¹¹
- If you access Claude via Google Vertex AI, Vertex’s documentation describes streaming responses using server-sent events. ¹²

Google Gemini - Gemini has strong long-context options and tool calling. - Gemini function calling is documented, and Gemini 2.0 Flash is described by Google as having built-in tool use and a 1M token context window in Vertex AI documentation. ¹³
- For realtime voice, Google provides the Gemini Live API for low-latency voice interactions, plus docs on audio formats and tool use in the Live API. ¹⁴

For this specific assignment, the governing constraint is the voice handoff and speed to ship. A pragmatic “best implementation” decision is: - Use a voice platform (Vapi or Retell) for telephony and voice agent runtime, because it compresses setup time, and keep your own scheduling and session state in your backend. - Use a strong text model for your orchestration and workflow logic, whichever provider you are most comfortable shipping quickly.

Voice architecture that preserves context across chat, outbound calls, and call-backs

You have two viable approaches, one optimised for speed, one for control.

Fastest: use an end-to-end voice agent platform

This aligns with the requirement not to build the entire voice stack from scratch.

Option A: Vapi - Vapi documents Twilio SIP trunking integration and provides outbound calling that you can initiate programmatically. ¹⁵

- The outbound API supports scheduling calls and initiating calls to numbers, which maps directly to your “Continue by phone” button. ¹⁶

- Vapi also documents HIPAA mode expectations and constraints in its security docs, which is relevant if you eventually handle PHI. ¹⁷

Option B: Retell - Retell provides Twilio deployment guidance and a documented approach for outbound calls, including an API for creating an outbound phone call. ¹⁸

- This also maps cleanly to “Call me now” and “call back to resume”, with your backend acting as the source of truth for state.

How context retention works with either - When starting a call, include in the initial agent context: - A 5 to 10 line summary of the web chat - Current structured workflow state (intake fields collected, selected provider, candidate slots) - The next question the agent should ask, for example “Confirm the Tuesday 10:30 slot with Dr X”

Retell’s docs emphasise you can “provide context to the agent” as part of building and deployment. ¹⁹

Bland AI is another outbound calling option and documents a “Send Call” endpoint that supports dynamic AI calls with objectives and actions, but you should verify compliance needs separately if you ever handle PHI. ²⁰

More control: Twilio Media Streams plus OpenAI Realtime

If you want to stand out technically and build your own bridge, this is the canonical approach: - Twilio Media Streams can stream raw audio from a call over WebSockets to a destination you specify, enabling real-time AI interaction. ²¹

- OpenAI Realtime provides low-latency audio in and out. ²²

- Twilio has published tutorials demonstrating outbound calls with Twilio Voice and Media Streams together with OpenAI Realtime. ²³

This approach is powerful but has more moving parts, which is why the end-to-end voice platforms are usually the best MVP decision.

The extra feature that is worth it: call-back continuity

Implement a single inbound phone number: - When a call comes in, Twilio sends a voice webhook to your server. ²⁴

- Your server looks up the most recent `conversation_id` for that caller’s phone number, then resumes the conversation from the saved summary and workflow state. - If the number is unknown, start a fresh intake or ask for a confirmation code.

This one feature dramatically improves perceived reliability and “product polish” without adding many UI surfaces.

Scheduling design with hard-coded providers and 30–60 day availability

The scheduling system should be deterministic. The AI can be human-like, but the slot logic must be exact.

Data model

Use a small set of types:

- Provider
- provider_id
- display_name
- body_part_focus (example: “knee”, “spine”, “shoulder”, “hand and wrist”)
- routing_terms (synonyms)
- visit_types supported (new patient, follow-up)
- default_visit_duration_minutes
- AvailabilitySlot
- slot_id
- provider_id
- start_time_iso
- end_time_iso
- visit_type
- is_booked
- Appointment
- appointment_id
- patient_id
- provider_id
- slot_id
- reason_text
- created_at
- channel (web chat or voice)
- Patient
- patient_id
- intake fields plus sms_opt_in boolean and timestamp

Hard-coded providers

Create four providers, each keyed to a body part:

- Knee and sports medicine
- Spine
- Shoulder and elbow
- Hand and wrist

Your matching step asks: “Which body part are you looking to be seen for?” and tries to map that to one provider.

Semantic matching approach

For a robust “semantic match” without overengineering, embeddings are the cleanest: - Precompute embeddings for each provider’s routing terms and short descriptions. - Embed the patient’s body-part phrase, then do cosine similarity against providers. - If top similarity is below a threshold, return “not treated” and offer a fallback.

OpenAI’s embeddings guide describes embeddings as turning text into numbers and enabling search and relatedness, which is exactly the mechanism you want for semantic routing. ²⁵

If you store embeddings in Postgres, pgvector is a common approach for vector similarity search and supports index types like HNSW for performance. ²⁶

If you prefer Redis, Redis documents vector search as well, but for an MVP, Postgres plus pgvector usually keeps the stack lean. ²⁷

The 30–60 day window and hard-coded availability

Given the current date of 2026-03-15 in America/New_York, “next 30–60 days” means roughly **2026-03-15 through 2026-05-14**.

To satisfy “hard-code availability” without maintaining hundreds of lines manually, a strong compromise is: - Commit a JSON file generated once (for example `availability_2026-03-15_to_2026-05-14.json`). - Generate it from a deterministic weekly template using a script, then treat the JSON as the source of truth during evaluation.

Example weekly templates: - Provider A: Mon and Tue mornings, Thu afternoons - Provider B: Wed all day - Provider C: Tue afternoons, Fri mornings - Provider D: Mon afternoons, Thu mornings

Then generate 30 or 45 minute slots.

Below is a small illustrative excerpt of what the committed JSON could look like (you would generate the full 60-day range in the repository):

```
{
  "timezone": "America/New_York",
  "providers": [
    {
      "provider_id": "prov_knee",
      "display_name": "Dr Morgan",
      "body_part_focus": "knee",
      "routing_terms": ["knee", "acl", "meniscus", "sports injury", "running injury"],
      "visit_types": ["new_patient", "follow_up"],
      "default_visit_duration_minutes": 30
    },
    {
      "provider_id": "prov_spine",
      "display_name": "Dr Patel",
      "body_part_focus": "spine",
      "routing_terms": ["back", "spine", "neck", "sciatica", "disc", "nerve"]
    }
  ]
}
```

```

    pain"],
    "visit_types": ["new_patient", "follow_up"],
    "default_visit_duration_minutes": 30
  },
  {
    "provider_id": "prov_shoulder",
    "display_name": "Dr Nguyen",
    "body_part_focus": "shoulder",
    "routing_terms": ["shoulder", "rotator cuff", "labrum", "frozen
shoulder", "impingement"],
    "visit_types": ["new_patient", "follow_up"],
    "default_visit_duration_minutes": 30
  },
  {
    "provider_id": "prov_hand",
    "display_name": "Dr Reyes",
    "body_part_focus": "hand and wrist",
    "routing_terms": ["hand", "wrist", "carpal tunnel", "trigger finger",
"arthritis"],
    "visit_types": ["new_patient", "follow_up"],
    "default_visit_duration_minutes": 30
  }
],
"slots": [
  {
    "slot_id": "slot_2026-03-17T10:00:00-04:00_prov_knee",
    "provider_id": "prov_knee",
    "visit_type": "new_patient",
    "start_time_iso": "2026-03-17T10:00:00-04:00",
    "end_time_iso": "2026-03-17T10:30:00-04:00",
    "is_booked": false
  },
  {
    "slot_id": "slot_2026-03-18T13:30:00-04:00_prov_spine",
    "provider_id": "prov_spine",
    "visit_type": "new_patient",
    "start_time_iso": "2026-03-18T13:30:00-04:00",
    "end_time_iso": "2026-03-18T14:00:00-04:00",
    "is_booked": false
  }
]
}

```

How the AI should propose times

For a good user experience, the algorithm should: - propose the next 3 to 5 slots automatically - show day-of-week labels in the patient's timezone - accept constraints like: - day of week, "Tuesday" - time of day, "after 3pm" - range, "sometime next week"

This can be implemented as a single deterministic filtering function, so the AI's role is only to interpret the constraint into a structured filter object.

Notifications, hosting, HTTPS, and safety and compliance guardrails

Email and SMS confirmations

Email can be implemented quickly with either AWS SES or SendGrid:

- AWS SES provides an API for sending messages and has standard “send email with SMTP or API” documentation. ²⁸
- SendGrid (Twilio) documents sending transactional email via SMTP or API. ²⁹

SMS should be optional and explicitly consent-based: - Twilio’s compliance documentation for A2P 10DLC states that anyone sending SMS/MMS over a 10DLC number to the US must register, and Twilio also emphasises consent practices. ³⁰

- Twilio’s messaging policy states that proof of opt-in consent should be retained per regulation or best practices. ³¹

In-product, treat SMS opt-in as a clear, separate step, and store `sms_opt_in_timestamp` and the wording shown to the patient.

Hosting on AWS EC2 with HTTPS

A common secure pattern is: - Run your app on EC2 instances behind an Application Load Balancer. - Use AWS Certificate Manager certificates on the ALB for TLS termination, AWS documentation recommends ACM for ALB certificates. ³²

- Configure the ALB HTTPS listener with the ACM certificate, AWS docs describe setting the Default SSL/TLS certificate sourced from ACM. ³³

For HIPAA-sensitive production work, AWS describes the Business Associate Addendum model for HIPAA workloads. ³⁴

Vendor compliance considerations

You can build the MVP without full HIPAA operationalisation, but you should design as if you will eventually handle PHI.

Key facts to anchor: - The US Department of Health and Human Services defines individually identifiable health information and notes it includes common identifiers like birth date. ¹

- Twilio states it will sign BAAs for HIPAA eligible products and notes that Programmable Voice and SMS can be HIPAA eligible. ³⁵

- OpenAI’s help centre describes how to obtain a BAA for API services, and OpenAI’s healthcare announcement notes eligible customers can apply for a BAA. ³⁶

Practically, your MVP should: - avoid storing freeform medical descriptions when not needed - log minimally, and redact phone and email in logs - treat conversation transcripts as sensitive - provide an operational “data retention” plan early

Testing and quality gates

For an assignment where “error-free” matters, bias toward automated tests for the core flows:

- Playwright is a widely used end-to-end testing framework for modern web apps and supports cross-browser testing. ³⁷
- Implement a Playwright test that:
 - books an appointment end-to-end
 - tests “Tuesday” constraint filtering
 - clicks “Continue by phone” and verifies the backend created a call request (mocked in CI)

For voice, implement “safety script” tests: - a list of prompts that attempt to elicit medical advice - required refusals and redirections - consistent phrasing and no hallucinated clinical claims

Recommended build order

To hit the deadline while still looking complete, the best order is:

- Build UI and chat transport first (streaming responses, typing indicators, polished glass UI).
- Implement scheduling workflow with hard-coded data and deterministic slot logic.
- Implement email confirmation.
- Implement voice handoff (outbound call) with the chosen voice platform, passing conversation summary and workflow state.
- Add call-back continuity as the single extra feature.
- Add office info workflow.
- Add minimal prescription refill intake workflow.

Implementation choices to standardise and hand off cleanly to another coding agent

If your goal is to hand this plan to another coding agent and start implementation immediately, use these principles:

- One canonical `ConversationSession` object in the database.
- Deterministic scheduling functions, the model never decides slot availability.
- Tool calling for every side effect (book, email, SMS, call).
- A strict JSON schema for tool arguments, to reduce errors, OpenAI function calling and structured outputs both support strictness patterns depending on whether you are calling tools or structuring the final response. ³⁸
- Prefer a voice platform API for speed, Vapi and Retell both document outbound calling and Twilio integration. ³⁹
- Store SMS consent and follow Twilio compliance guidance for opt-ins and registration if you send US SMS via 10DLC. ⁴⁰

Finally, keep your vendor names abstracted behind interfaces. That way, if you prototype voice with one platform, you can swap later without rewriting the workflow logic.

Key external services to treat as first-class entities in your system design are Amazon Web Services ⁴¹ for hosting and security controls, Twilio ⁴² for SMS and telephony primitives, and OpenAI ⁴³, Anthropic ⁴⁴, and Google ⁴⁵ for LLM and realtime model options. For voice platforms that accelerate

time-to-demo, Vapi ⁴⁶ and Retell AI ⁴⁷ are both documented with Twilio integration and outbound calling capabilities. ³⁹

- 1 Summary of the HIPAA Privacy Rule
https://www.hhs.gov/hipaa/for-professionals/privacy/laws-regulations/index.html?utm_source=chatgpt.com
- 2 ⁴⁵ Glassmorphism: Definition and Best Practices
https://www.nngroup.com/articles/glassmorphism/?utm_source=chatgpt.com
- 3 Motion Documentation (prev Framer Motion) - Motion.dev
https://motion.dev/docs?utm_source=chatgpt.com
- 4 Streaming - App Router
https://nextjs.org/learn/dashboard-app/streaming?utm_source=chatgpt.com
- 5 ³⁸ ⁴⁴ Function calling | OpenAI API
https://developers.openai.com/api/docs/guides/function-calling/?utm_source=chatgpt.com
- 6 Structured model outputs | OpenAI API
https://developers.openai.com/api/docs/guides/structured-outputs/?utm_source=chatgpt.com
- 7 Agents SDK | OpenAI API
https://developers.openai.com/api/docs/guides/agents-sdk/?utm_source=chatgpt.com
- 8 ⁴¹ AI Risk Management Framework
https://www.nist.gov/itl/ai-risk-management-framework?utm_source=chatgpt.com
- 9 GPT-5.4 Model | OpenAI API
https://developers.openai.com/api/docs/models/gpt-5.4?utm_source=chatgpt.com
- 10 Voice agents | OpenAI API
<https://developers.openai.com/api/docs/guides/voice-agents/>
- 11 ⁴⁶ Tool use with Claude - Claude API Docs
https://platform.claude.com/docs/en/agents-and-tools/tool-use/overview?utm_source=chatgpt.com
- 12 Anthropic's Claude models | Generative AI on Vertex AI
https://docs.cloud.google.com/vertex-ai/generative-ai/docs/partner-models/claude?utm_source=chatgpt.com
- 13 Function calling with the Gemini API | Google AI for Developers
https://ai.google.dev/gemini-api/docs/function-calling?utm_source=chatgpt.com
- 14 Gemini Live API overview - Google AI for Developers
https://ai.google.dev/gemini-api/docs/live-api?utm_source=chatgpt.com
- 15 Twilio SIP Integration | Vapi
<https://docs.vapi.ai/advanced/sip/twilio>
- 16 ³⁹ Outbound Calling
https://docs.vapi.ai/calls/outbound-calling?utm_source=chatgpt.com
- 17 HIPAA Compliance
https://docs.vapi.ai/security-and-privacy/hipaa?utm_source=chatgpt.com
- 18 ¹⁹ ⁴⁷ Twilio - Retell AI
<https://docs.retellai.com/deploy/twilio>
- 20 Send Call - Bland AI
<https://docs.bland.ai/api-v1/post/calls>

21 **Media Streams Overview**

https://www.twilio.com/docs/voice/media-streams?utm_source=chatgpt.com

22 42 **Realtime API**

https://developers.openai.com/api/docs/guides/realtime/?utm_source=chatgpt.com

23 **Make Outbound calls with Python, the OpenAI Realtime ...**

https://www.twilio.com/en-us/blog/outbound-calls-python-openai-realtime-api-voice?utm_source=chatgpt.com

24 **Voice Webhooks**

https://www.twilio.com/docs/usage/webhooks/voice-webhooks?utm_source=chatgpt.com

25 **Vector embeddings | OpenAI API**

https://developers.openai.com/api/docs/guides/embeddings/?utm_source=chatgpt.com

26 **pgvector/pgvector: Open-source vector similarity search for ...**

https://github.com/pgvector/pgvector?utm_source=chatgpt.com

27 43 **Vector search concepts | Docs**

https://redis.io/docs/latest/develop/ai/search-and-query/vectors/?utm_source=chatgpt.com

28 **Using the Amazon SES API to send email - AWS Documentation**

https://docs.aws.amazon.com/ses/latest/dg/send-email-api.html?utm_source=chatgpt.com

29 **Getting Started with Transactional Email | SendGrid Docs**

https://www.twilio.com/docs/sendgrid/for-developers/sending-email/getting-started-with-transactional-emails?utm_source=chatgpt.com

30 40 **Programmable Messaging and A2P 10DLC**

https://www.twilio.com/docs/messaging/compliance/a2p-10dlc?utm_source=chatgpt.com

31 **Twilio Messaging Policy**

https://www.twilio.com/en-us/legal/messaging-policy?utm_source=chatgpt.com

32 **SSL certificates for your Application Load Balancer**

https://docs.aws.amazon.com/elasticloadbalancing/latest/application/https-listener-certificates.html?utm_source=chatgpt.com

33 **Create an Application Load Balancer - AWS Documentation**

https://docs.aws.amazon.com/elasticloadbalancing/latest/application/create-application-load-balancer.html?utm_source=chatgpt.com

34 **HIPAA Compliance - Amazon Web Services (AWS)**

https://aws.amazon.com/compliance/hipaa-compliance/?utm_source=chatgpt.com

35 **Programmable Voice, SIP, and SMS are now HIPAA Eligible**

https://www.twilio.com/en-us/changelog/programmable-voice--sip--and-sms-are-now-hipaa-eligible?utm_source=chatgpt.com

36 **How can I get a Business Associate Agreement (BAA) with ...**

https://help.openai.com/en/articles/8660679-how-can-i-get-a-business-associate-agreement-baa-with-openai?utm_source=chatgpt.com

37 **Playwright: Fast and reliable end-to-end testing for modern ...**

https://playwright.dev/?utm_source=chatgpt.com